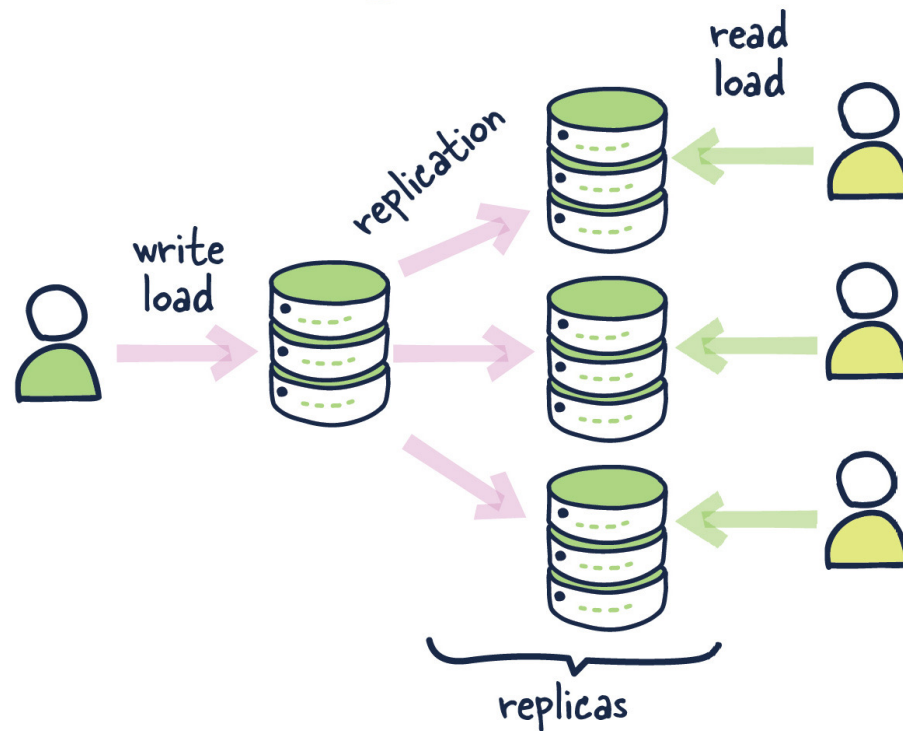


# La réplication avec MongoDB

## Scaling Reads



# La réplication avec MongoDB

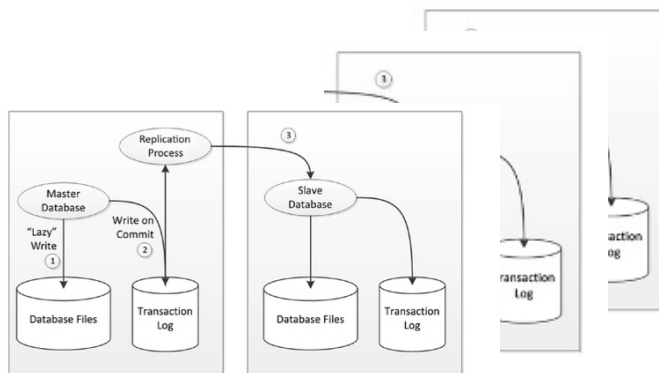
La réplication est particulièrement adaptée aux applications ayant une **forte charge en lecture**.

Elle permet de :

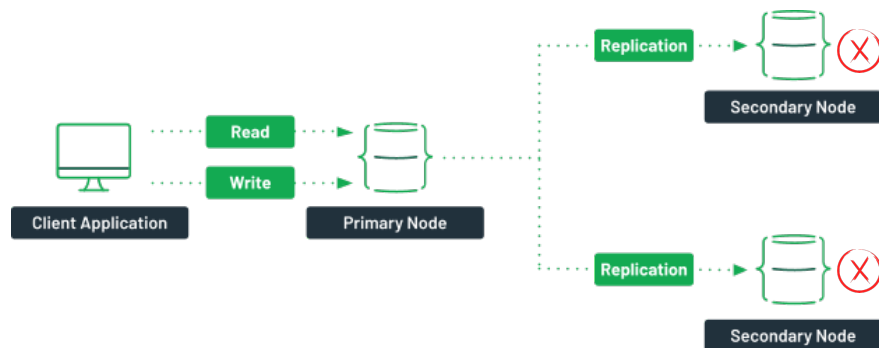
- **répartir les lectures** sur plusieurs nœuds secondaires ;
- **augmenter la scalabilité** en lecture ;
- **améliorer la disponibilité** grâce à plusieurs copies des données.

La réplication implique cependant un compromis entre **performance et cohérence** : plus on exige de garanties lors des écritures, plus la latence peut augmenter. À l'inverse, une réplication asynchrone peut conduire à lire temporairement des **données plus anciennes sur un secondaire**.

**Scaler les lectures implique donc de choisir le bon compromis entre performance, disponibilité et cohérence.**



# La gestion de la cohérence : Eventual consistency



Facteur de réplication FR : 3

**Validation en écriture W : 1**

Le système confirme l'écriture dès que l'écriture sur le primaire est validée et la réplication sur les 2 nœuds restants se fait de manière asynchrone.

Facteur de réplication FR : 3

**Validation en lecture R : 1**

Le système renvoie le résultat à partir de n'importe quel nœud. Résultat éventuellement incohérent.

Facteur de réplication FR : 3

**Validation en lecture R : 2**

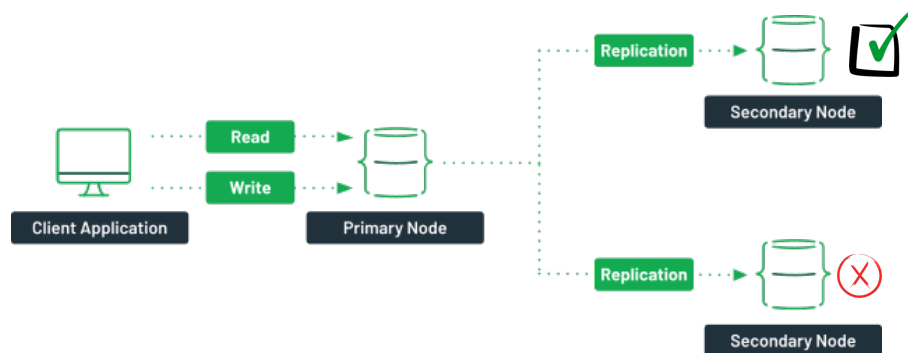
Le système renvoie le résultat en se mettant d'accord sur le résultat avec un second nœud. Résultat éventuellement incohérent.

Facteur de réplication FR : 3

**Validation en lecture R : 3**

Le système renvoie le résultat en se mettant d'accord entre les 3 nœuds. Résultat cohérent

# La gestion de la cohérence : Eventual consistency



Facteur de réplication FR : 3

**Validation en écriture W : 2**

Le système confirme l'écriture dès que l'écriture sur le primaire est validée et que la réplication est validée sur au moins 1 réplica.

**La cohérence des données lues est établie selon le niveau de cohérence demandé en écriture :**

$$W + R > FR$$

Facteur de réplication FR : 3

**Validation en lecture R : 1**

Le système renvoie le résultat à partir de n'importe quel nœud.  
Résultat éventuellement incohérent.

Facteur de réplication FR : 3

**Validation en lecture R : 2**

Le système renvoie le résultat en se mettant d'accord sur le résultat avec un second nœud.  
Résultat cohérent.

Facteur de réplication FR : 3

**Validation en lecture R : 3**

Le système renvoie le résultat en se mettant d'accord entre les 3 nœuds.  
Résultat cohérent



# La réplication avec MongoDB

- ❑ Pour configurer un cluster en mode réplication, il sera nécessaire de déclarer les membres de votre cluster.
  
- ❑ Comment le primaire est élu ?
  - L'élection se fait avec un mécanisme de vote à la majorité (quorum)
  - Uniquement les membres ayant une carte de vote (vote:1) et une priorité (priority > 0) peuvent voter.
  - Pour définir des membres à privilégier comme primaire il suffit d'associer une priorité plus élevée que les autres membres.
  - Un membre avec une priorité de 0 ne peut pas devenir primaire mais il contiendra une copie des données.
  - Les membres avec un vote à 0 ne contribueront pas à l'élection du primaire.
  
- ❑ **Les rôles pour les membres :**
  - Primary : priority > 0 et vote =1
  - Secondary : priority > 0 et vote =1
  - Hidden: idem 'Secondary' mais non visible (vote=0 et priority=0). Il contient des données et peut être utilisé pour les backups.
  - Arbiter : utilisé lors du mécanisme d'élection du primaire (vote=1 et priority=0), il ne contient pas de données.

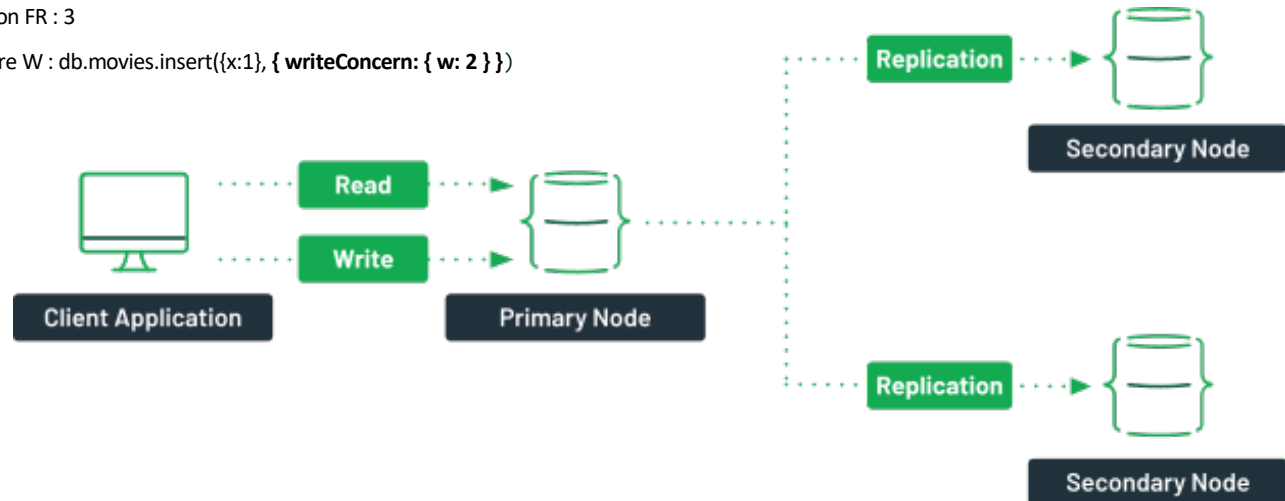
# La réplication avec MongoDB

La cohérence des données lues est établie selon le niveau de cohérence demandé en écriture :

$$W + R > FR$$

Facteur de réplication FR : 3

Validation en écriture W : `db.movies.insert({x:1}, { writeConcern: { w: 2 } })`



**Lecture cohérente**  
Validation en lecture R : 2  
`db.movies.find().readConcern(majority)`



**Lecture éventuellement incohérente**  
Validation en lecture R : 1  
`db.movies.find().readConcern(local)`

## Chap 6 : MongoDB\_Replicaset\_TP01

